

Technical Documentation: Smart IoT Fire & Smoke Detection System

Author: Sarah Gamal Mohamed

1. Introduction

Fire and smoke accidents represent significant risks to human life, property, and critical infrastructure. Early detection of abnormal environmental conditions—such as sudden temperature spikes, the presence of combustible gases, or open flames—can drastically reduce response times and mitigate potential damage.

This document details the design, architecture, and implementation of an **IoT-based Fire & Smoke Detection System**. Utilizing the ESP32 microcontroller as its core processing unit, the system continuously monitors environmental parameters. Upon detecting hazardous conditions, it autonomously triggers local alarms, activates ventilation systems, and updates a real-time web dashboard for remote monitoring.

2. System Architecture

The system is designed with a modular architecture, separating sensing, processing, actuation, and communication layers. This ensures reliability and facilitates future scalability.

2.1 Hardware Components

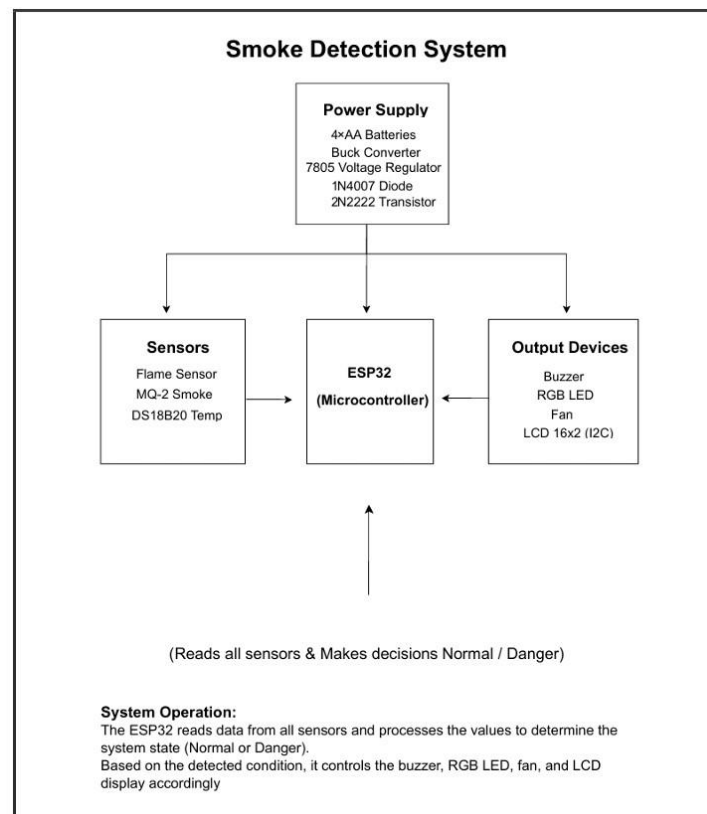
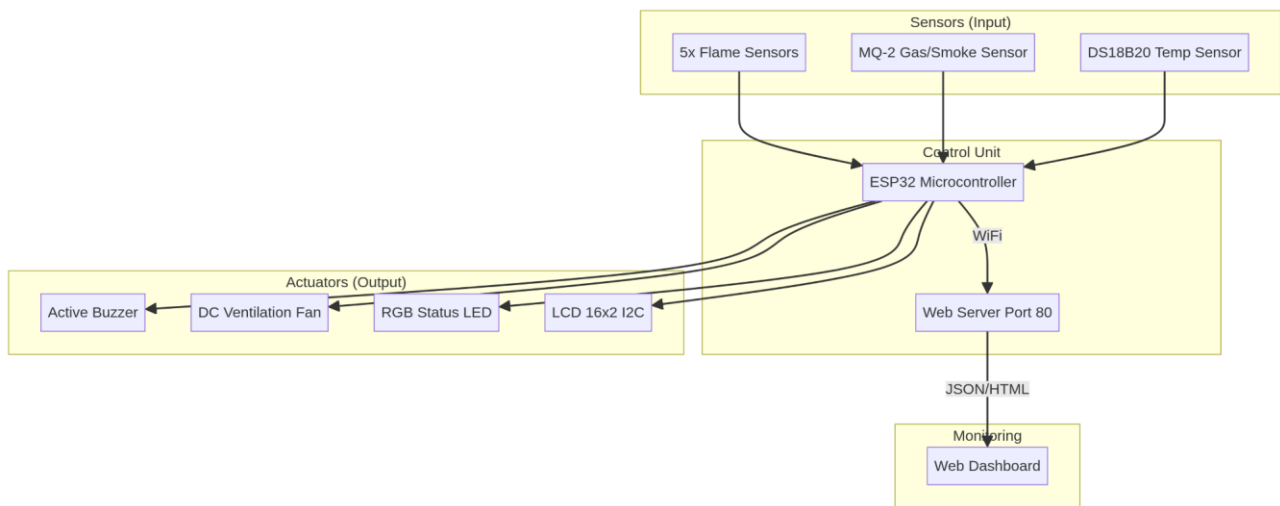
The hardware selection prioritizes rapid response times and robust connectivity. The following table outlines the primary components utilized in the system:

Category	Component	Technical Specifications & Role
----------	-----------	---------------------------------

Microcontroller	ESP32 Development Board	Dual-core processor with integrated Wi-Fi. Acts as the central control unit, processing sensor data and hosting the web server.
Input Sensor	Flame Sensors (x5)	Infrared (IR) receivers positioned to provide 360-degree coverage. They output digital signals upon detecting the specific IR wavelength emitted by flames.
Input Sensor	MQ-2 Gas/Smoke Sensor	Analog sensor highly sensitive to LPG, propane, hydrogen, and smoke. Used to detect pre-combustion conditions.
Input Sensor	DS18B20 Temperature Sensor	Digital thermometer communicating via the OneWire protocol, providing high-precision ambient temperature readings.
Actuator	DC Fan with Motor Driver	A ventilation fan controlled via Pulse Width Modulation (PWM) to exhaust smoke. Speed varies based on hazard severity.
Actuator	Active Buzzer	Provides a high-decibel audible alert during critical danger states.
Interface	LCD 16x2 (I2C)	Local display unit showing real-time sensor values and system status, utilizing the I2C protocol to minimize GPIO pin usage.
Power Supply	4xAA Batteries & Buck Converter	Provides stable 5V and 3.3V power rails to the ESP32 and peripherals.

2.2 System Block Diagram

The data flow begins at the sensor layer, where environmental metrics are captured and transmitted to the ESP32. The microcontroller evaluates this data against predefined safety thresholds. Based on the logic state, it commands the output devices (buzzer, fan, LED) and simultaneously updates the web interface.



3. Implementation Logic and Workflow

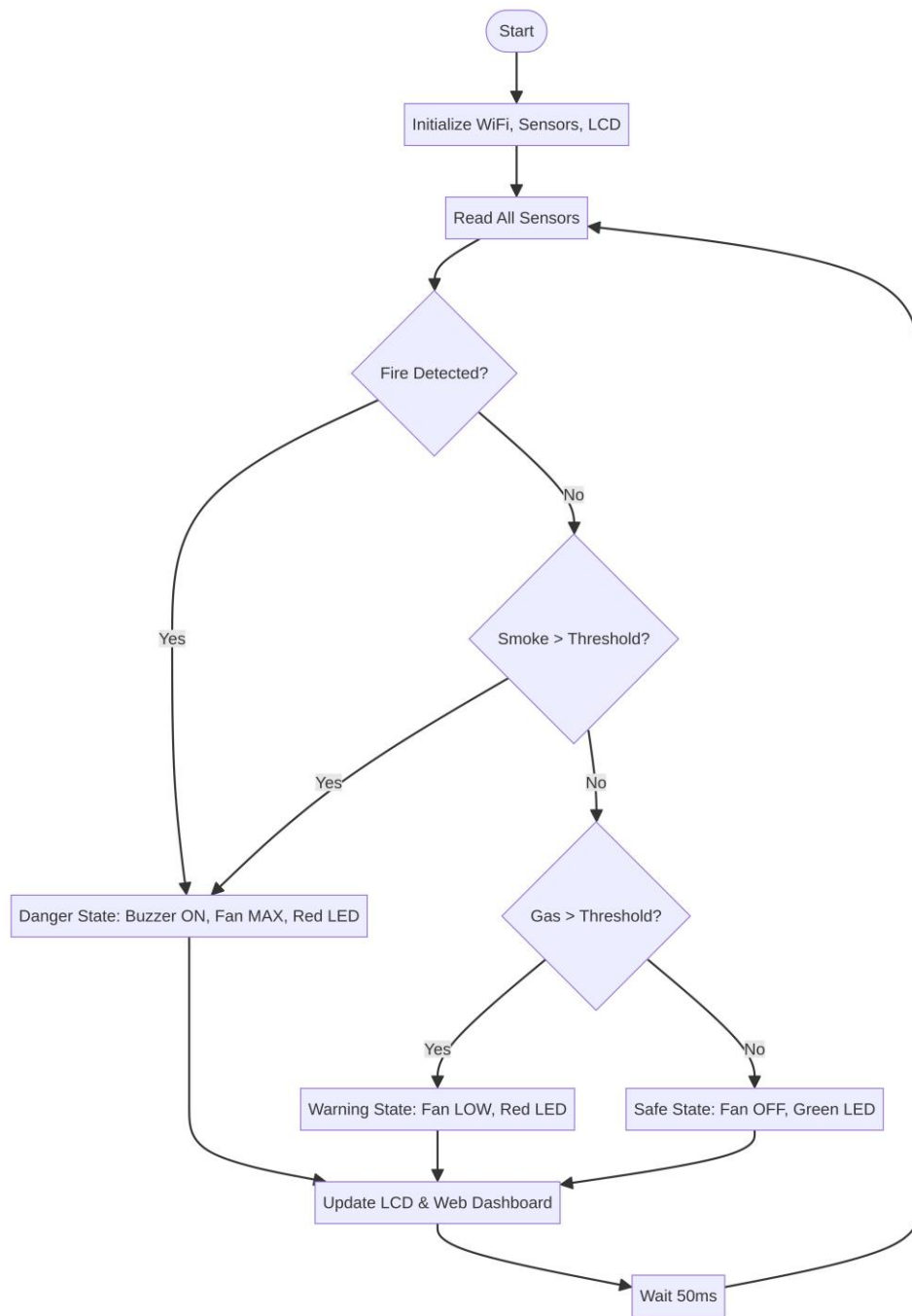
The core logic of the system is designed to operate in a continuous loop, polling sensors every 50 milliseconds to ensure near-instantaneous hazard detection. The system categorizes the environment into three distinct states.

3.1 Operational States

1. **Safe State:** Under normal conditions, no fire or smoke is detected. The system maintains a passive monitoring role. The RGB status LED illuminates green, the ventilation fan remains off, and the buzzer is silent.
2. **Warning State:** If the MQ-2 sensor detects gas or smoke concentrations exceeding the low-level threshold (but below critical levels), the system enters a warning state. The RGB LED shifts to red, and the ventilation fan is activated at a low speed to begin clearing the air.
3. **Danger State:** This critical state is triggered if any of the five flame sensors detect a fire, or if the smoke concentration exceeds the high-level threshold. The system immediately activates the buzzer, turns the RGB LED red, and drives the ventilation fan at maximum speed to exhaust smoke rapidly.

3.2 Logic Flowchart

The decision-making process executed by the ESP32 is illustrated in the flowchart below:



4. Software and Connectivity

The software implementation leverages the Arduino framework for the ESP32. It combines low-level hardware control with high-level networking capabilities.

4.1 Web Server and Dashboard

The ESP32 hosts an asynchronous web server on port 80. This server provides a responsive, mobile-friendly dashboard accessible to any device on the local network.

The dashboard features:

- **Real-Time Updates:** Utilizing AJAX, the web page fetches new data every two seconds without requiring a full page reload.
- **Visual Indicators:** Color-coded alerts (Green for Safe, Red for Danger) provide immediate visual feedback regarding the system’s status.
- **Data Endpoint:** The system exposes a JSON endpoint (`/data`), which structures the sensor readings. This allows for easy integration with external cloud platforms or mobile applications in the future.

5. Testing and Validation

The system underwent rigorous testing to validate its responsiveness and reliability under simulated hazard conditions.

Test Scenario	Expected Outcome	Actual Result	Status
Baseline Monitoring	System remains in Safe State; Green LED active.	Maintained stable operation with accurate temperature readings.	PASS
Smoke Introduction	Transition to Warning State; Fan activates at low speed.	MQ-2 detected smoke rapidly; fan engaged as programmed.	PASS
Flame Exposure	Immediate transition to Danger State; Buzzer and Fan at MAX.	Flame sensors triggered instantly; alarms activated without latency.	PASS
Network Stability	Continuous web dashboard updates without connection drops.	Dashboard refreshed every 2 seconds consistently over a 24hour period.	PASS

6. Conclusion and Future Scope

The IoT-based Fire & Smoke Detection System successfully demonstrates a reliable, automated approach to environmental safety. By integrating multi-sensor fusion with the processing power of the ESP32, the system provides both immediate local mitigation and remote monitoring capabilities.

Future Enhancements:

- **Cloud Integration:** Transitioning from a local web server to a cloud-based platform (e.g., AWS IoT or Firebase) to enable global monitoring and historical data logging.
- **Push Notifications:** Implementing mobile alerts via services like IFTTT or Telegram to notify users instantly, regardless of their location.
- **Active Suppression:** Expanding the actuation layer to include relay-controlled water pumps or automated fire extinguishers for active hazard suppression.